

Frontend - Arquitetura e regras de negocio

Projeto SPA-Vue - documento gerado em 21/05/2026

Visao geral

O frontend fica em `project_form` e foi preparado como uma SPA em Nuxt 3. O Nuxt usa Vite no ambiente de desenvolvimento e no build, Pinia para estado global e Vitest para testes unitarios.

A aplicacao conversa com o backend `Backend_API` por HTTP usando JWT. O token recebido no login e salvo pelo store de autenticao e enviado nas proximas requisicoes pelo plugin de API.

Front

Nuxt 3, Vite, Pinia, TypeScript, Vitest e CSS global.

Back

ASP.NET Core, Entity Framework Core, SQL Server, JWT Bearer e FluentValidation.

Tecnologias

- Nuxt 3 com `ssr: false`.
- Vite como bundler do Nuxt.
- Pinia e `@pinia/nuxt` para estado global.
- Vitest, Vue Test Utils e Happy DOM para testes.
- TypeScript em modo estrito.
- GitHub Actions para build, testes, geracao estatica e deploy no GitHub Pages.

Estrutura principal

- `pages/login.vue` : entrada do usuario, autenticao JWT e troca da senha padrao apos o primeiro login.
- `pages/alterar-senha.vue` : tela autenticada para alterar senha a qualquer momento.
- `pages/usuarios/novo.vue` : cadastro de usuario sem campo de senha.
- `pages/diretoria/novo.vue` : cadastro da tabela Diretoria.
- `stores/auth.ts` : sessao, usuario logado, token, flag de senha padrao e acao de alteracao de senha.
- `middleware/auth.global.ts` : protecao de rotas e bloqueio de navegacao enquanto a senha padrao nao for alterada.
- `components/PasswordStrengthMeter.vue` : medidor visual de forca de senha.
- `utils/password-strength.ts` : regra reutilizavel para classificar senhas.

Regra de negocio - cadastro de usuario

O cadastro de usuario nao exige mais o campo de senha. A tela envia apenas dados cadastrais, como nome, email, telefone e perfil.

Novos usuarios recebem automaticamente a senha inicial **Senha@252525**.

Essa senha não é salva em texto puro. O backend aplica hash PBKDF2-SHA256 pelo `PasswordHasher`.

Regra de negocio - troca obrigatoria

Quando o usuário faz login, o backend valida email e senha e devolve o JWT. Na mesma resposta, também devolve a flag abaixo:

```
{
  "deveAlterarSenhaPadrao": true
}
```

Se essa flag vier como `true`, o frontend mantém o usuário autenticado, mas bloqueia a navegação normal e exibe a tela de troca de senha.

O usuário deve informar senha atual, nova senha e confirmação. A nova senha deve ser diferente da senha atual e não pode continuar sendo `Senha@252525`.

Depois que a senha é alterada com sucesso, o Pínia limpa a flag `deveAlterarSenhaPadrao` e libera a navegação.

Medidor de força de senha

O medidor usa `utils/password-strength.ts`. A classificação considera tamanho mínimo, letras minúsculas, letras maiúsculas, números e caracteres especiais.

As faixas exibidas são: não informada, fraca, media e forte. O componente `PasswordStrengthMeter.vue` mostra barra visual, rótulo da força e dicas para melhorar a senha.

Autenticacao e estado

O store `stores/auth.ts` concentra token JWT, dados resumidos do usuário, data de expiração, indicador `deveAlterarSenhaPadrao` e ações `login`, `logout`, `fetchMe` e `alterarSenha`.

A sessão é persistida no `localStorage` para sobreviver ao recarregamento da página. O middleware global usa esse estado para proteger rotas autenticadas.

Integracao com o backend

- `POST /api/Auth/login`
- `GET /api/Auth/me`
- `POST /api/Auth/alterar-senha`
- `POST /api/usuarios`
- `POST /api/Diretoria`

O endpoint de alteração de senha exige JWT válido.

Protecao contra SQL injection

No backend, as consultas usam Entity Framework Core com LINQ. Os valores de entrada são enviados como parâmetros pelo provider do EF Core, evitando concatenação manual de SQL.

Também foram adicionadas validações com FluentValidation nos contratos de entrada.

Para prevenir regressão, existe um teste automatizado em

`Backend_API/form_API.Tests/Security/SqlInjectionProtectionTests.cs`. Esse teste falha se APIs de SQL bruto forem introduzidas no código da aplicação, como `FromSqlRaw`, `ExecuteSqlRaw` ou `SqlQueryRaw`.

Testes

```
cd project_form
npm test
```

```
cd Backend_API
dotnet test form_API.Tests/form_API.Tests.csproj
```

Foram adicionadas coberturas para store de autenticação, troca de senha no Pinia, medidor de força de senha, regra backend da senha padrão, validação da nova senha e proteção contra SQL bruto.

CI/CD do frontend

O workflow `.github/workflows/frontend-pages.yml` executa um job específico para o front:

1. checkout do repositório;
2. setup do Node.js 22;
3. `npm ci`;
4. `npm test`;
5. `npm run generate`;
6. upload do artefato estático;
7. deploy no GitHub Pages quando não for pull request.

Como o frontend está em uma subpasta do mesmo repositório, o job usa `working-directory: project_form`.

Para GitHub Pages, `NEXT_APP_BASE_URL` é definido como `/{nome-do-repositorio}/`. A URL da API deve ser configurada em `NEXT_PUBLIC_API_BASE` nas variáveis ou secrets do repositório.

Como rodar localmente

```
cd Backend_API
dotnet run
```

```
cd project_form
npm ci
$env:NEXT_PUBLIC_API_BASE='http://localhost:5001/api'
npm run dev
```

Para gerar a versao estatica, execute `npm run generate` dentro de `project_form`.